

User-Centric MEC: 협력 AP와 분산 오프로딩

Towards Decentralized Task Offloading and Resource Allocation in User-Centric MEC

Langtian Qin, Hancheng Lu, Yuang Chen, Baolin Chong, Feng Wu (2023)

UCMEC에서 오프로딩·전력·CPU 자원을 함께 결정하는 두 PPO 방식

문제의 시작: 무선 품질과 계산 지연은 함께 움직인다

약한 access link는 오프로딩 자체를 불리하게 만들고, 협력 수신은 새 fronthaul 부담을 만든다

01 셀 가장자리 fading과 inter-cell interference가 rate를 낮춘다



02 user-centric cluster 사용자마다 여러 AP가 uplink를 함께 받는다



03 wireless fronthaul AP가 받은 신호를 CPU로 전달해야 한다



04 공동 제어 offload·power·CPU share가 delay를 함께 바꾼다

좋은 access rate만으로 끝나지 않는다. fronthaul과 CPU 자원까지 같은 결정을 통해 연결된다

UCMEC의 실제 데이터 경로

AP는 RF 신호와 channel estimation을 맡고, CPU는 decoding과 task processing을 맡는다

01 사용자 task 각 slot에 data size와 cycles/bit가 주어진다



02 AP cluster 선택된 여러 AP가 collaborative uplink를 수신



03 mmWave fronthaul cluster의 신호를 선택 CPU로 전달



04 CPU + MEC signal decoding 뒤 computing resource를 배분

‘AP에서 계산’ 모델이 아니다. AP와 CPU의 역할 및 access·fronthaul 주파수는 분리된다

slot마다 누가 무엇을 결정하는가

학습 agent는 discrete offload·power를, CPU는 연속 computing share를 담당한다

결정	주체	형태	핵심 제약
AP cluster	system model	사용자별 X_i 개 AP	cluster size는 X_i
오프로딩	사용자 agent	CPU k 선택 또는 local	한 사용자는 최대 한 CPU
uplink power	사용자 agent	P^a 의 discrete level	최대 전력 집합 안
CPU share c_{mk}	선택 CPU	연속 allocation	한 CPU의 합은 C_k^e

전력·destination은 RL action, CPU share는 고정 action 아래의 convex subproblem이다

지연은 local 또는 offload branch에서 생긴다

한 task는 분할하지 않는다. local 또는 access→fronthaul→edge 중 선택한 branch의 지연을 본다

구간	계산	느려지는 원인	논문 식
local	$D \times \text{cycles/bit} \div \text{local CPU}$	작은 local capacity	Eq. (18)
access	$D \div \text{uplink rate}$	fading·uplink interference	Eq. (19)
fronthaul	가장 느린 AP link 기준	AP-CPU rate와 interference	Eq. (20)
edge	$D \times \text{cycles/bit} \div c_{mk}$	CPU share 경쟁	Eq. (21)
total	active branch delay	local 또는 offload 선택	Eqs. (22)-(23)

offload를 택한 경우 access가 빨라도 fronthaul·CPU 병목으로 지연이 커질 수 있다

원문 최적화 문제 P1: 긴 시간 평균 delay 최소화

한 slot만 빠르게 하는 action이 아니라, 매 slot 동적 channel-task 상태에서 평균 total delay를 줄인다

01 목적 $(M \times T)$ 로 평균한 모든 사용자의 total delay 최소화



02 binary choice ω_{mk} 는 0 또는 1, user당 최대 한 CPU



03 resource feasibility CPU share 합·power set·cluster size를 지킨다



04 deadline T_m 이 coherence time을 넘으면 task를 discard

P1은 binary·discrete·continuous 변수가 delay와 rate를 통해 엮힌 nonconvex MINLP다

해법의 분해: RL이 이산 선택, convex solver가 CPU share

논문은 P1 전체를 closed-form으로 푸는 것이 아니라, 관측된 action 아래의 allocation을 분리한다

- | | | |
|----|------------|---|
| 01 | state 관측 | task size·density와 과거 action·delay를 입력 |
| ↓ | | |
| 02 | PPO action | 각 user가 offload destination과 power를 선택 |
| ↓ | | |
| 03 | P2/P4 | 고정 action에서 CPU resource allocation을 푼다 |
| ↓ | | |
| 04 | feedback | 실현 delay로 reward와 다음 state를 만든다 |

convex solver는 P1 전체가 아니라, 고정 action 아래의 CPU allocation을 푼다

비협력 모드 IPPO: 다른 사용자를 환경으로 본다

정보 공유가 어렵거나 malicious behavior 가능성이 있을 때, 각 user가 자기 PPO를 독립 학습한다

- | | | |
|----|--------------------|---|
| 01 | local observation | 자기 task D·density ρ ·직전 action을 본다 |
| ↓ | | |
| 02 | actor action | CPU 선택과 discrete data power를 낸다 |
| ↓ | | |
| 03 | system response | 간섭·fronthaul·CPU share가 delay를 결정 |
| ↓ | | |
| 04 | independent update | 각 user가 자기 reward로 PPO를 갱신 |

IPPO는 분산 action을 내지만, peer policy 변화가 내 환경을 바꾸는 비정상성을 남긴다

MAPPO: 전체로 훈련, local actor로 실행

node collaboration이 가능하고 malicious behavior 확률이 낮다는 조건에서 CTDE를 사용한다

- | | | |
|----|-----------------------|---|
| 01 | 정보 공유 | 각 user observation과 previous action을 모은다 |
| ↓ | | |
| 02 | central critic | training 때 global state와 joint action을 평가 |
| ↓ | | |
| 03 | actor update | CPU가 업데이트한 actor parameter를 user에게 보낸다 |
| ↓ | | |
| 04 | distributed execution | online에는 각 user가 local actor로 action을 낸다 |

MAPPO의 centralization은 critic training에 한정된다. 실행 action은 각 local actor가 낸다

IPPO와 MAPPO는 무엇이 다른가

같은 UCMEC control problem에 대해, 정보 공유 가능성에 따라 training architecture가 달라진다

항목	IPPO	MAPPO	해석 경계
전제	non-cooperative	collaboration 가능·low malicious probability	성능 순위의 보편 법칙 아님
학습 정보	각 user의 local observation	central critic의 global state·joint action	execution input과 구분
실행 action	각 user actor	각 user local actor	MAPPO도 centralized execution 아님
offline cost	완전 분산 training	global information 처리	Fig. 4의 조건부 training-time 비교

협력은 더 넓은 학습 정보를 쓰지만 offline training cost를 높인다

계산 trace 예시: 언제 offload path가 더 빠른가

K=1로 한정된 별도 계산: 같은 task의 두 binary action을 각각 평가한다

action	active branch	계산	Eq. (23) total
task	K=1, D=2 Mbit	2 Mbit × 500	1.0e9 cycles
local omega=0	T_l only	1.0e9 ÷ 0.5e9	2.00 s
offload omega=1	T_a+T_f+T_e	0.10+0.05+0.50	0.65 s
comparison	same task	2.00-0.65	offload 1.35 s faster

K=1 toy에서 offload action은 local보다 1.35초 빠르다

실험은 어떤 환경에서 비교됐는가

그래프의 차이는 특정 simulator, task range, AP·CPU 구성과 PPO setting에 조건부다

범주	원문 설정	학습 설정	주의
coverage	900 × 900 m ² , 3 CPUs	users·AP 수를 sweep	현장 deployment 측정이 아님
compute	CPU 10-20 GHz, local 2-5 GHz	task 50- 100 KB	density 500- 1000 cycles/bit
wireless	access 1.9 GHz, 20 MHz	maximum power 100 mW	채널·fronthaul model 가정 포함
training	300 steps/episode, 2 × 64 hidden layers	PPO lr 5e-4, gamma .99	offline simulator training

수치적 우위는 이 simulator 조건의 평균 성능이며 다른 환경에 자동 이식되지 않는다

Fig. 4: convergence와 training time

10 users, 50 AP 설정에서 PPO 계열은 약 3000 episodes 부근에서 안정화되는 모습을 보인다

01	Proposed / CBO / MPO	PPO policy clipping으로 비교적 안정적인 curve
↓		
02	IQL / MADDPG	원문 그림에서 더 큰 fluctuation을 보인다
↓		
03	IPPO	MAPPO와 유사한 convergence behavior를 보인다
↓		
04	CTDE cost	global critic 때문에 MAPPO의 offline training이 더 길다

그림은 이 구현의 learning behavior이지, 모든 workload의 convergence 보장은 아니다

Fig. 5: access 협력의 rate 이득은 sweep마다 다르다

Proposed (Coop)의 최대 비교 이득은 원문이 서로 다른 축과 baseline에서 보고한 범위다

sweep	고정 조건	원문 최대 rate 이득	해석
사용자 수	AP=50	7.31%-208.76%	사용자가 늘면 interference가 커져 rate가 감소
AP 수	users=10	12.88%-326.99%	더 많은 serving AP가 fading 효과를 완화
maximum power	users=10, AP=50	8.13%-227.94%	feasible power range가 넓어짐
cluster size	CBO는 clustering 없음	10.05%-343.56%	diversity gain과 비교 조건을 함께 봐야 함

343.56%는 전체 평균이 아니라 Fig. 5 cluster-size sweep의 최대 비교 값이다

Fig. 6: AP·power·cluster에는 모두 trade-off가 있다

협력은 delay를 낮출 수 있지만, 더 많은 AP 또는 cluster가 언제나 같은 폭으로 이득을 주지는 않는다

관찰	원문 조건부 결과	원문 해석	경계
AP 수 증가	최대 19.59%-45.57% delay 감소	access와 fronthaul delay를 낮춤	configured model의 sweep
power 증가	0.2 W 이후 추가 delay gain 포화	rate가 올라가도 한계효용 감소	100 mW 기본 setting 과 구분
cluster 증가	최대 16.31%-38.24% delay 감소	access 이득, fronthaul request·interference 증가	diminishing gain
사용자 증가	최대 8.33%-35.06% 감소 비교	CPU·fronthaul 경쟁이 커짐	절대 delay가 낮아진다는 뜻 아님

협력 access 이득은 fronthaul overhead를 상쇄할 수 있지만, cluster 확대 이득은 줄어든다

이 논문이 보여 주지 않는 것

교육용으로 method의 범위와 실험적 주장 사이를 분명히 구분한다

논문이 다룬 범위	이 텍에서 말하지 않는 것	이유
simulated UCMEC uplink	실제 field deployment throughput 보장	평가는 channel-task model 기반 simulation
offline PPO training	online safety-deadline guarantee	deadline penalty가 formal guarantee는 아님
fixed Xi AP cluster model	cluster selection 자체의 optimal policy	P1의 cluster size는 Xi constraint
cooperation conditions	악의적 node가 있는 환경의 robustness	MAPPO는 low malicious probability 조건을 둠

기여는 UCMEC simulator의 joint control design이며, 넓은 deployment guarantee는 아니다

핵심: wireless와 compute를 한 loop에

UCMEC의 핵심은 AP 협력만이 아니라, 그로 인해 생긴 fronthaul·CPU 경쟁까지 관측과 reward에 반영하는 것이다

01	문제	cellular MEC의 약한 access link와 edge effect
↓		
02	구조	user-specific AP cluster → fronthaul → CPU
↓		
03	결정	PPO의 offload·power와 convex CPU allocation
↓		
04	조건	IPPO 또는 CTDE MAPPO를 collaboration 가능성에 맞춤

결과는 조건부지만, access·fronthaul·edge를 한 control loop에 넣은 것이 핵심이다